Summative Project

Text Classification for an African Language: Sentiment Classification of Amharic Tweets

Mahlet Tilahun
October 18, 2026

Project links:

- **GitHub repository**: https://github.com/mahlet-tilahun/amharic-tweet-sentiment-classifier
- **Demo video:** https://youtu.be/Q2jRcuwaXWc?si=Y4gRbVThzgZuqNjh
- **Live app:** https://amharic-tweet-sentiment-classifier.streamlit.app/

1. Abstract

The objective of this project is to develop a 3-class sentiment classifier for Amharic tweets using the AfriSenti formal benchmark. Firstly, classical baseline models were developed, followed by BiLSTM and two pre-trained Transformer models. In light of the above observation, the optimal model turned out to be an Amharic BERT fine-tuning, which achieved a test macro F1 score of 0.6249 and a weighted F1 score of 0.7249. The main challenge in this context was the problem of label shift between the training/dev set and the test set since the dominant class had changed from neutral to negative. Additionally, homophone normalization and error analysis were performed to account for difficult examples including Amharic words which were unseen during training, code-mixed tweets and class imbalance. In this regard, the final framework will be effectively provided via Streamlit web application in order to use the model interactively.

2. Introduction

Sentiment analysis automatically identifies if the provided text is negative, neutral, or positive. There are good sentiment analysis tools available for English, but most of the African languages still lack labeled data sets and models (Joshi et al., 2020). Amharic is one of such languages. It is the working language of the government of Ethiopia; it uses its own Ge'ez (Fidel) writing system, and it is used by Ethiopians extensively in social media (Yimam et al., 2020). Automated sentiment analysis of Amharic posts can enable journalists and media observers to monitor public sentiments about the news and assist NGOs and researchers in understanding public sentiments, and also help businesses understand the feedback left by their customers in Amharic.

Amharic poses various challenges to Natural Language Processing for the following three reasons that are relevant to this study. The first one is the presence of a rich morphology. Amharic has one word with affixes that indicate person, negation, possession, and prepositions. For instance, ለ·በሰሽ ("you

(fem.) having worn") and አታነቢም ("you (fem.) don't read") are single words. This increases the number of rare word types; hence, word-based models are likely to encounter unseen words.

Secondly there are spelling variations. There are several Fidel characters that have similar pronunciations (ሀ/ሐ/ኀ, ሰ/ሠ, አ/ዐ, ጸ/ፀ), hence, the same word could have various orthographic forms (Belay et al., 2022).

Last but not least, there is the varies use of social media language. Tweets include sarcasm, figurative expressions, English words, and Amharic written in Latin script. Yimam et al. (2020) noted that these were the reasons for errors in Amharic sentiment analysis.

The main objective of this project will be to develop a 3-class (negative/neutral/positive) sentiment classifier for Amharic tweets using real human-annotated data. This will be done following the steps of establishing baselines, testing if RNN (BiLSTM) and transformers pretrained on Amharic perform better, determining the reasons behind poor performance of the models using error analysis, and finally making the model deployable through a web application.

Research questions

1. Does homophone normalization and character-level data help address the problem of variation and morphology in Amharic?
2. Does a pretrained transformer outperform the classical baseline model and the BiLSTM model that was trained from scratch in the presence of 6,000 training tweets?

3. Related Work

On Amharic sentiment analysis, initial research relied on lexicon and classical classifiers trained on limited domain-specific data. Yimam et al. (2020) developed Amharic Sentiment Annotator Bot (ASAB) tool for sentiment annotation of Amharic messages on Telegram and collected around 9.4k Amharic tweets annotated by three annotators each, comparing TF-IDF classifiers, embeddings and FLAIR models. F1 score of the best TF-IDF +

Logistic Regression model varied between 37% and 53%, and their best model was a FLAIR classifier using distributional-thesaurus embeddings. Their findings showed that sarcasm, figurative language, mixed scripts and typos were the biggest challenges.

Another related work is AfriSenti, released by Muhammad et al. (2023a), a sentiment analysis benchmark for Twitter with about 110 thousand tweets in 14 African languages, annotated by native speakers, including Amharic data used here. The same data was used for the SemEval-2023 Task 12 (Muhammad et al., 2023b). All strong participants fine-tuned multilingual Transformers, mostly AfroXLMR. The winning Amharic model (Alam et al., 2023) fine-tuned AfroXLMR-large using phylogeny-based adapters, additional training data and ensembling, and achieved a weighted F1 score of 78.4 on Amharic test split.

Pretrained models for African languages include XLM-R (Conneau et al., 2020), which is a multilingual Transformer that has been pretrained on 100 languages, including Amharic. Alabi et al. (2022) designed AfroXLMR through the process of continued pretraining (multilingual adaptive fine-tuning, MAFT) on 17 African languages and high resource languages. Also, a smaller version of AfroXLMR called AfroXLMR-mini is designed from MiniLM model that has been distilled from XLM-R (Wang et al., 2020). There are monolingual models in Amharic like BERT-medium-Amharic (rasyosef, 2024) based on BERT architecture (Devlin et al., 2019) that have been pretrained on roughly 290M Amharic tokens.

Regarding addressing the gap, the best AfriSenti architectures available are big (AfroXLMR-large, 560M parameters) and employ ensembles and additional datasets, therefore making them costly to train and to deploy. There is scarcity of comparative researches done using small architectures and only a limited number of explanations of why Amharic language models do not perform well on this particular dataset. In this project's case, I observed that the test dataset had a completely different distribution of classes compared to train/dev sets. This project provides a controlled comparison of inexpensive baseline architectures, an LSTM and two small transformers (≤120M parameters).

4. Dataset and Data Preparation

4.1 Source and licence

The source of data is the Amharic component of the AfriSenti corpus (Muhammad et al., 2023a) from the official shared task repository (afrisenti-semeval/afrisent-semeval-2023, data/amh folder). The license is CC BY 4.0. The tweets are naturally occurring Twitter messages annotated by native Amharic speakers, without any synthetically generated or machine-generated data. User mentions were pre-anonymised as @user by the dataset providers.

4.2 Size, splits and class definitions

I retained the train/dev/test splits provided by the authors to compare my results with the literature and make sure there are no overlapping tweets (0 overlaps checked). The classes were defined according to the data description: positive (the author expresses a positive opinion/emotion), negative (negative opinion/ criticism/insult/anger) and neutral (factual or no sentiment at all).

split	tweets	negative	neutral	positive
train	5,984	1,548 (25.9%)	3,104 (51.9%)	1,332 (22.3%)
dev	1,497	388 (25.9%)	776 (51.8%)	333 (22.2%)
test	1,999	1,337 (66.9%)	224 (11.2%)	438 (21.9%)

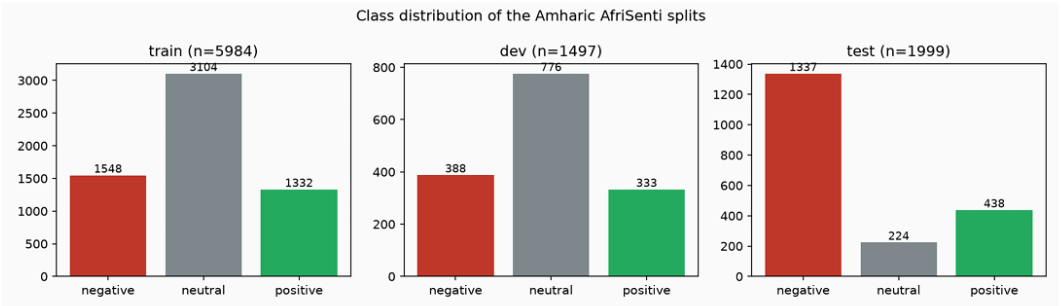


Figure 1. Class distribution per split.

4.3 Properties and quality problems encountered during exploration

As can be seen from the analysis performed in `src/explore_data.py` and stored in `results/data_stats.json`, tweets are short, averaging 14-15 words with the longest tweet containing 44 words. The training vocabulary is huge and sparse since 25,394 (74%) out of 34,412 unique words occur only once. Also, about 30% of development samples and 34% of test samples are out of vocabulary(OOV), and homophone normalisation decreases the number of OOV words by 0.7% in development and 0.6% in test.

The distribution of labels is also quite different: 52% of training and development samples are neutral, while test samples are 67% negative and 11% neutral, which makes the test dataset biased towards negative samples. Additionally, there are differences in format: 62% of test tweets have `@user`, and neither test nor development samples contain any emojis or "..." symbol, and tweets are quoted, while 3.4% of training tweets have emojis, and "..." occurs 1,635 times.

Code-mixing is exhibited in all the splits; roughly 10% of training and development tweets, and 15% of test tweets have words in the Latin alphabet, which may include English or romanised Amharic. What is also noticeable is that there are 2 duplicate tweets in train, with no conflicting duplicate texts.

4.4 Preprocessing

The same clean_text() function was used for all models (also deployed in the web app, ensuring consistency between training and deployment). The steps include the following:

1. Remove quote marks (test file), @user, URLs, and the # symbol (hashtag word retained);
2. Replace Ethiopic punctuation (federalis, federalism, federalist) with spaces, and reduce repeating !!!, ...;
3. Lowercase Latin characters (Amharic letters (Fidel) do not have capitalisation);
4. Homophones normalization: map every vowel order combination of ሐ and ኀ to ሀ, ሠ to ሰ, ዐ to አ and ፀ to ጸ. Each Fidel consonant family requires 8 consecutive Unicode characters, and the normalization

function is implemented using Unicode character arithmetic (utils.py) and includes all 7 possible vowel orders;

5. Remove extra whitespaces.

Stop words or emoji removal was not performed, as emojis can convey emotion, and I did not implement stemming as no dependable Amharic stemmer was available. The emoji removal was also tried as an experiment (B6).

5. Methodology.

Each model makes predictions on one of three categories and is trained only on train. Every decision regarding hyperparameters, best epoch and best model is done on dev and test data is used only once per experiment for the final figures.

5.1 Baselines (scikit-learn)

B0 to B6 represent experimental IDs in the results table to represent the baselines. “B” represents the baselines and the number is a unique representation of a model or change in the model. Baseline B0 majority-class baseline which classifies all classes as neutral and acts as the lower bound. B1 and B2 make use of word unigram and bigram using sub-linear TF-IDF weighting (Salton & Buckley, 1988) and multinomial logistic regression with C = 5. B1 does not do homophone normalization while B2 does this task, this answers the first research question.

B4 is character 2–5-grams within the boundaries of the word (char_wb, min_df=2) using TF-IDF vectorization with logistic regression. Since the Amharic language words are quite long and extensively inflected, the character n-grams could be similar to the stem or affixes of unknown words such as the negative word አል…ም. B5 is similar to B4 but also includes the class-balanced loss function (class_weight="balanced"). B6 is like B5 but also deletes emojis and typographic symbols in order to make the training text resemble the test text.

5.2 BiLSTM (PyTorch, from scratch)

Tokens in the input are assigned ID numbers, whereby tokens appearing less than twice are mapped to <unk>. Input sequences are padded up to 64 tokens. This neural network is made up of an embedding layer, whose size is 128 and which is randomly initialized and trained, a dropout layer at 0.3, and a one-layer bidirectional LSTM with 128 hidden units for each direction. The output will be max-pooled with time padding mask and dropped out prior to feeding into the fully connected layer of dimension 3.

The training involves using cross-entropy with equal class weights, Adam with learning rate of 1e-3, batch size of 32, gradient clipping of 1.0, and up to 15 epochs. Early stopping is employed if development macro-F1 does not increase for 3 consecutive epochs. The following tokenization schemes are tested: L_word tokenization employs whitespace tokens, with vocabulary size of 8,919 and 1.4 million parameters, while L_subword involves AfroXLMR SentencePiece Tokenizer, with vocabulary size of 3,288 tokens which occurred at least twice, and 0.69 million parameters. We can note that only the tokenizer is used here and not the pretrained weights.

5.3 Finetuned Transformers (Hugging Face)

Pretrained Transformer encoder (Vaswani et al., 2017) converts the tokenized tweet into context representations via self-attention mechanism. First token's vector representation is passed through a randomly initialized classification layer (dense + tanh + linear to 3 classes). All parameters are updated. I compare two small models of a similar computational cost:

	AfroXLMR-mini (Alabi et al., 2022)	BERT-medium-Amharic (rasyosef, 2024)
type	multilingual (17 African + 3 high-resource languages)	monolingual Amharic
origin	MiniLM distilled from XLM-R-large, then MAFT	BERT pretrained from scratch on ~290M Amharic tokens
layers / hidden / heads	12 / 384 / 12	8 / 512 / 8
vocabulary	250,002 SentencePiece	28,672 WordPiece
parameters	117.6M (96M of them in the embedding matrix)	40.5M

Fine-tuning configuration (same for both models): max length 64 tokens (the mean tweet contains 35 AfroXLMR tokens and only 2.7% have more than 64 tokens), batch size 16, AdamW (learning rate 5e-5, weight decay 0.01), linear learning rate schedule with 10% warm-up, gradient norm clipping 1.0, 4 epochs, cross-entropy loss with balanced class weights, and the model checkpoint with the highest F1 score on the dev set is selected. Training loop code was developed from scratch using pure PyTorch code that encompassed the forward pass, the loss computation, backpropagation, updating of parameters, and updating of the scheduler, instead of using the Trainer class. This provided clarity in all the steps involved. The training was done using an 8-core CPU without a GPU which had an impact on model sizes.

5.4 Metrics for evaluation

The Macro-F1 score is used as the primary metric since it measures the average F1 score of the classes without any weighting applied, which implies equal weight on all three sentiments. It is helpful because the dominating sentiment shifts from neutral in the development dataset to negative in the testing dataset. In addition to that, accuracy, macro precision, and macro recall are calculated to give more insights into the results and see if the errors are due to over- or under-prediction of some classes. The weighted F1 score is used because it is the official metric used by AfriSenti-SemEval and is used to compare with the published related work. Finally, the model's errors will be analyzed using the confusion matrices and per-class scores.

6. Experiments and Results

The complete set of experiments is provided in results/experiments.csv. The best baseline was the class-balanced character TF-IDF + logistic regression classifier with test macro-F1 = 0.4608 and test weighted F1 = 0.5397. The BiLSTM using subword tokens performed worse than the basic character baseline (0.4522) on test, demonstrating that a small LSTM network, having been trained on just several thousand examples, is insufficient to outperform strong n-gram features. On the other hand, both transformer models outperformed the baseline.

Experiment	Dev macro-F1	Test macro-F1	Test weighted F1
B0 majority class	0.2276	0.0672	0.0226
B5 char TF-IDF + balanced LogReg	0.5312	0.4608	0.5397
L_bilstm_subword	0.4807	0.4522	0.5452
T1 AfroXLMR-mini	0.5371	0.5795	0.7057
T2 BERT-medium-Amharic	0.6038	0.6249	0.7249

The chosen model for the task is T2 BERT-medium-Amharic. It performed the best among all other models in terms of macro-F1 score on dev and test datasets, as well as weighted F1 score. This means that monolingual Amharic model is more suitable for the domain than multilingual AfroXLMR-mini despite being smaller.

7. Error Analysis and Discussion

The primary problem with this model is that it does not have problems with noise; instead, the neutral class is being over-predicted. As it can be seen from the confusion matrix, in the case of the final model, 273 negative tweets were classified as neutral, and 104 negative tweets were classified as positive. It is

difficult for the model to classify tweets into the neutral class since many tweets are short and do not have any emotional words; they just say some fact or opinion.
From the analysis of errors, it can be seen that the difficulty of the examples depends on the number of out-of-vocabulary words. In the case when the ratio is low, accuracy is always above 0.70; however, when the ratio is high, accuracy decreases to 0.68. This confirms the initial hypothesis about the morphological and vocabulary sparsity problem in Amharic social-media text. Tweets written in code-mixed language are also harder since the model does not have enough training data for the English words in the Amharic context. This reiterates the fact that many Amharic tweets on social media are code-mixed.

Class shift is another key factor at play. The train/dev set is biased towards neutrality while the test set is biased towards negativity. The model learns a bias towards neutral on training over the train distribution. Even the optimal model cannot distinguish between negative and neutral tweets since it is a challenging task and distribution is mismatched. This is the key reason for the poor macro-F1 of < 0.70 despite high raw F1.

8. Deployment

The last classifier is implemented as a web application using the Streamlit library. The public version of the application can be accessed on Amharic Tweet Sentiment Classifier. The web page allows users to input or paste their Amharic tweet along with choosing Classify tweet button to get predictions in form of negative, neutral, or positive sentiments and probabilities for each class. Besides, the page shows the processed text along with warnings if an input is not composed of the Ge'ez characters.

The Streamlit application interacts with the fine-tuned Amharic BERT model directly. When users submit their text, the web application uses the clean_text() preprocessing function that was used during the training process, tokenizes the output using the trained model tokenizer, and gets class probabilities through inference. The model and tokenizer are loaded from the hosted version of the model repository on Hugging Face, which allows the web application to work

with the fine-tuned model without storing the big size of the model files in the GitHub repository.

This deployment presents an implementation that helps one test the classifier, although it needs to be understood that errors could be made by the classifier, especially if there are ambiguities of sentiment, code-mixed text, and tweets deviating from the training dataset. Links for the deployment and the model are included in the project's README file. The online implementation of the project can be accessed here.

9. Limitations and Future Work

There are several limitations for this project. Firstly, the dataset used in this project is relatively small, consisting of only about 6,000 train examples. Secondly, the test set is distribution shifted and does not follow the same distribution as the train dataset, making it difficult to compare it fairly in one-shot. Thirdly, the project does not use a large, state-of-the-art multilingual or monolingual model due to limitations of CPU-only training and time constraints. Lastly, some social media posts may include sarcastic or code-switched phrases, making them difficult to predict with a small model. Potential works for the future can be experimenting well with data augmentation, cross-domain calibration for dealing with class shift, and using larger pretrained model on a GPU.

10. Conclusion

In this project, we have seen that simple baselines can be useful but small pretrained Transformers outperform them on the sentiment analysis dataset of Amharic language. The best model - BERT-medium-Amharic - scored great results of 0.6249 in test macro F1 and 0.7249 in test weighted F1, outscoring the balanced n-grams baseline and BiLSTM. In conclusion, we identified some of the prominent difficulties encountered when carrying out Amharic Natural Language Processing tasks, which show the necessity of building an efficient pre-processing pipeline and reliable model.

References

- Alabi, J. O., Adelani, D. I., Mosbach, M., & Klakow, D. (2022). Adapting pre-trained language models to African languages via multilingual adaptive fine-tuning. In Proceedings of the 29th International Conference on Computational Linguistics (COLING 2022) (pp. 4336–4349). International Committee on Computational Linguistics. https://aclanthology.org/2022.coling-1.382/

- Alam, M. M. I., Xie, R., Faisal, F., & Anastasopoulos, A. (2023). GMNLP at SemEval-2023 Task 12: Sentiment analysis with phylogeny-based adapters. In Proceedings of the 17th International Workshop on Semantic Evaluation (SemEval-2023) (pp. 1172–1182). https://doi.org/10.18653/v1/2023.semeval-1.163

- Belay, T. D., Tonja, A. L., Kolesnikova, O., Yimam, S. M., Ayele, A. A., Haile, S. B., Sidorov, G., & Gelbukh, A. (2022). The effect of normalization for bi-directional Amharic-English neural machine translation. arXiv:2210.15224. https://arxiv.org/abs/2210.15224

- Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., & Stoyanov, V. (2020). Unsupervised cross-lingual representation learning at scale. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (pp. 8440 8451). https://doi.org/10.18653/v1/2020.acl-main.747

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of NAACL-HLT 2019 (pp. 4171–4186). https://doi.org/10.18653/v1/N19-1423

- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735 1780. https://doi.org/10.1162/neco.1997.9.8.1735
- Joshi, P., Santy, S., Budhiraja, A., Bali, K., & Choudhury, M. (2020). The state and fate of linguistic diversity and inclusion in the NLP world. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (pp. 6282–6293). https://doi.org/10.18653/v1/2020.acl-main.560
- Muhammad, S. H., Abdulmumin, I., Ayele, A. A., Ousidhoum, N., Adelani, D. I., Yimam, S. M., et al. (2023a). AfriSenti: A Twitter sentiment analysis benchmark for African languages. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (pp. 13968–13981). https://doi.org/10.18653/v1/2023.emnlp-main.862
- Muhammad, S. H., Abdulmumin, I., Yimam, S. M., Adelani, D. I., Ahmad, I. S., Ousidhoum, N., Ayele, A. A., Mohammad, S., Beloucif, M., & Ruder, S. (2023b). SemEval-2023 Task 12: Sentiment analysis for African languages (AfriSenti-SemEval). In Proceedings of the 17th International Workshop on Semantic Evaluation (SemEval-2023) (pp. 2319–2337). https://doi.org/10.18653/v1/2023.semeval-1.315
- rasyosef. (2024). bert-medium-amharic [Pretrained language model]. Hugging Face. https://huggingface.co/rasyosef/bert-medium-amharic .
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0
- Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. IEEE Transactions on Signal Processing, 45(11), 2673–2681. https://doi.org/10.1109/78.650093
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In Advances in Neural Information Processing Systems 30 (pp. 5998 6008).
- Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In Advances in Neural Information Processing Systems 33 (pp. 5776–5788).
- Yimam, S. M., Alemayehu, H. M., Ayele, A., & Biemann, C. (2020). Exploring Amharic sentiment analysis from social media texts: Building

annotation tools and classification models. In Proceedings of the International Conference on Computational Linguistics (pp. 1048–1060). https://doi.org/10.18653/v1/2020.coling-main.91